Project Status Report

Blockchain-Based Lightweight Decentralized Secure Communication (Nexus)

Project: Nexus Community
Date: June 30, 2026
Document Type: Implementation Gap Analysis & Feasibility Report

---

Executive Summary

Nexus is a **hybrid** communication platform: a Discord-style centralized stack (PostgreSQL, Clerk, Socket.IO, UploadThing, LiveKit) with an **optional WebRTC P2P layer** for live text delivery. Several proposal features are implemented; others exist only partially or not at all. The architecture is **not fully decentralized** today — it is better described as **server-assisted P2P** with blockchain logging as an add-on.

Category	Count
Fully implemented	6
Partially implemented	5
Not implemented	6

---

1. Feature-by-Feature Status

1.1 Novelty Features (Core Proposal)

#	Proposed Feature	Status	Current Implementation
1	Decentralized Communication	**Partial**	WebRTC P2P mesh (`packages/p2p/`), signaling relay (`apps/signaling-server/`). History, auth, files, and A/V still use central servers.
2	Command Propagation / Alternate Routing	**Not implemented**	Socket.IO room broadcast + P2P fan-out only. No multi-hop relay or automatic path failover.
3	Incremental Chunk-Based File Transfer	**Not implemented**	Whole-file uploads via UploadThing CDN. No P2P chunked transfer.
4	Secure Communication (AES-GCM + SSL/TLS)	**Partial**	AES-256-GCM at rest and on P2P wire. TLS is deployment-level (HTTPS/WSS), not certificate-based peer auth.
5	Performance Monitoring & Analytics	**Not implemented**	Basic `/health` and connection indicators only. No latency, bandwidth, or packet analytics.

1.2 Add-On Features

#	Proposed Feature	Status	Current Implementation

1	Blockchain Security	**Partial**	Client logs message hashes to external node at `BLOCKCHAIN_URL`. No blockchain node in this repo.
2	Multimedia Support	**Implemented**	Images, PDF, video, audio via UploadThing; voice/video via LiveKit.
3	Secure Hidden Messages (Steganography)	**Implemented**	LSB steganography + optional AES password layer.
4	Suspicious Message Detection	**Partial**	Keyword list flags for blockchain log only; no blocking or user alerts.
5	Self-Destructing Messages	**Not implemented**	Soft delete exists; no TTL, auto-expiry, or burn-after-read.
6	Fake Node Detection	**Not implemented**	Peers identified by random UUID; no trust scores or Sybil resistance.

1.3 Additional Proposal Claims

Claim	Status	Notes
Certificate-based authentication	Not implemented	Clerk JWT/session auth instead
Lightweight consensus mechanism	Not in repo	Depends on external blockchain service
Energy-efficient resource management	Not measured	No metrics or optimization layer
Scalable large-scale device connectivity	Partial	WebRTC mesh does not scale well beyond small groups
Eliminates single point of failure	Not achieved	Postgres, Clerk, API, UploadThing, LiveKit are SPOFs

---

2. What Is Remaining (Implementation Gap)

2.1 High Priority (Directly Maps to Proposal)

1. **True decentralized messaging** — Local/offline history (IndexedDB, CRDT), libp2p for NAT traversal, reduce dependence on PostgreSQL for live chat.
2. **Incremental chunk-based P2P file transfer** — Split files into chunks over WebRTC data channels with resume, checksum per chunk, and reassembly.
3. **Command propagation with alternate routing** — Multi-hop relay when direct P2P fails; route selection and failover logic.
4. **Performance monitoring dashboard** — Latency, packet loss, bandwidth, peer count metrics.
5. **Self-destructing messages** — Schema needs `expiresAt` or TTL; background job or client-side timer; optional burn-after-read for DMs.

6. **Fake node detection** — Trust/behavior scoring per peer; reputation from message patterns and failed handshakes.
7. **Blockchain node in-repo** — Lightweight PoW or PoA chain embedded in monorepo (today: external service only).
8. **Certificate-based authentication** — X.509 or WebRTC DTLS fingerprint verification; mTLS between peers or API.

2.2 Medium Priority (Strengthen Existing Features)

9. **Suspicious message detection (complete pipeline)** — Block/quarantine messages, admin alerts, optional ML/heuristics.
10. **Complete call signaling** — Frontend has call events; backend Socket.IO handlers appear incomplete.
11. **P2P encryption key exchange** — Per-session keys instead of shared `ENCRYPTION_MASTER_KEY`.
12. **Blockchain verification UI** — Verify hash on-chain from client; chain explorer for audit trail.

2.3 Lower Priority (Polish & Documentation)

13. Energy efficiency benchmarks and lightweight consensus documentation
14. Deployment guide for TLS/WSS in production
15. Academic test results (latency, throughput, security analysis)

---

3. What Can Be Added More (Feasible Enhancements)

Tier A — Achievable in 2–4 Weeks

Enhancement	Effort	Value for Report
Self-destructing messages (TTL field + cron/UI timer)	Medium	Directly covers Add-On #5
Performance metrics panel (WebRTC stats API)	Medium	Covers Novelty #5
Suspicious message blocking + admin notification	Low	Completes Add-On #4
Embedded minimal blockchain node	Medium	Makes Add-On #1 self-contained
Chunk upload for large files (server-side)	Medium	Partial credit for Novelty #3
Message integrity verification (hash vs blockchain)	Low	Strengthens blockchain story

Tier B — Achievable in 1–2 Months

Enhancement	Effort	Value for Report
P2P chunked file transfer over WebRTC	High	Full Novelty #3
IndexedDB local message cache for offline P2P	Medium	Stronger decentralization
Peer reputation / basic fake-node scoring	High	Add-On #6
DTLS fingerprint display and manual peer verification	Medium	Partial cert-based trust

Multi-hop relay via signaling server	High	Novelty #2
End-to-end encryption where server cannot decrypt	High	Stronger privacy narrative

Tier C — Good Differentiators for Viva/Demo

- QR-based peer pairing for offline secure channel setup
- Steganography in video/audio frames
- Blockchain-backed file hash registry
- Read receipts with blockchain timestamp
- Network topology visualization (live P2P mesh graph)
- Exportable security audit report (PDF)

---

4. What Is Not Possible or Not Feasible

4.1 Architecturally Infeasible (Without Major Redesign)

Claim	Why It Is Not Feasible As Stated
Fully decentralized with zero central servers	Auth (Clerk), DB (Postgres), files (UploadThing), A/V (LiveKit) are centralized. Full decentralization needs IPFS, libp2p, DHT, and decentralized identity.
Blockchain on every message at scale	PoW per message is slow and energy-heavy. Hash logging (current approach) is the right lightweight alternative.
SSL/TLS + certificate auth for P2P peers in browser	Browsers use WebRTC DTLS, not custom X.509 client certs. True cert-based P2P auth needs native apps or custom PKI.
Eliminate all single points of failure	Hybrid architecture always has SPOFs (DB, auth, signaling).
Large-scale mesh (1000+ peers per channel)	WebRTC is $O(n^2)$ connections. Discord-scale needs SFU/MCU (like LiveKit), not pure mesh.

4.2 Technically Difficult (Beyond Typical UG/PG Timeline)

Feature	Challenge
Perfect fake node detection	Sybil attacks need economic stake, web-of-trust, or centralized identity
ML-based malicious message detection	Needs dataset, training pipeline, false-positive handling
Energy-efficient lightweight consensus	Real consensus (Raft, PBFT, PoS) is complex; toy chain is demo-only
Steganography in all media types	LSB in images works; video/audio steg is fragile under compression
True E2EE with server-blind storage	Key management, recovery, multi-device sync — Signal-level complexity

4.3 Misleading if Claimed Without Evidence

Claim	Risk
"Tamper-proof communication"	Server can still modify DB; only hashes on blockchain are tamper-evident
"No dependence on centralized servers"	Current system depends heavily on central infra
"Fault-tolerant at scale"	Not demonstrated; no load tests or failover tests in repo
"Lightweight consensus"	No consensus implementation visible in codebase

4.4 Recommended Report Wording

> *"A hybrid secure communication framework combining centralized persistence and authentication with optional peer-to-peer real-time delivery, AES-GCM encryption, blockchain-backed message integrity logging, and steganographic covert channels — designed as a stepping stone toward full decentralization."*

---

5. Implementation Completion Summary

Module	Completion
Encryption (AES-GCM)	~90%
Decentralized P2P	~40%
Blockchain integration	~35%
File transfer (chunk-based)	~10% (CDN only)
Steganography	~85%
Suspicious detection	~25%
Self-destruct messages	0%
Fake node detection	0%
Performance monitoring	~5%
Certificate authentication	0%
Overall (weighted)	**~35–40%**

---

6. Current Architecture

```
Browser (Next.js)
  ├── REST/Socket.IO ──► Node API (Hono) ──► PostgreSQL (encrypted messages)
  │                          └── optional ──► External blockchain (localhost:5001)
  ├── WebRTC P2P mesh ──► Signaling relay (ws://:3001) [SDP/ICE only]
  ├── UploadThing ──► CDN file hosting
  └── LiveKit ──► Voice/video SFU
```

Proposed vs Actual

Proposed	Actual
Fully P2P Mesh	Optional WebRTC P2P (partial)
Blockchain Node	External API client only
Chunk File Transfer	UploadThing CDN only
Cert Auth	Clerk JWT auth
No Central Server	Postgres + Node API + Clerk

---

7. Recommended Priority Roadmap (Before Viva/Demo)

1. Embed a minimal blockchain node in the monorepo
2. Self-destructing messages (high visibility, clear privacy story)
3. Performance metrics widget (latency, peers, bandwidth)
4. Suspicious message alert in UI (not only console/blockchain log)
5. Document hybrid architecture honestly in final report

---

8. Conclusion

Strengths already in the codebase: AES-GCM encryption, WebRTC P2P mesh, LSB steganography with password protection, multimedia via LiveKit and UploadThing, and blockchain hash logging with suspicious keyword flagging.

Main gaps vs the proposal: chunk-based P2P file transfer, alternate route propagation, performance analytics, self-destructing messages, fake node detection, certificate-based authentication, and an in-repo blockchain/consensus layer.

Feasible path forward: Complete Tier A enhancements, document the hybrid model clearly, and position full decentralization as future work (libp2p, IPFS, CRDT) rather than claiming it is already done.

Keywords: Decentralized Communication, Blockchain, Secure Messaging, Peer-to-Peer, Cryptography, Gap Analysis, Feasibility Study

Prepared for: Blockchain based Lightweight Decentralized Secure Communication Project
Supervisor: Mrs. Apurva Hujare
Head of Department: Prof. Abhijeet More